Chapter 2

WHAT IS SOFTWARE QUALITY

Please note that author slides have been significantly modified

Chapter 2
What is software quality?

- Outline
  - What is software?
  - Software errors, faults and failures
    - differences
  - Classification of the causes of software errors
  - Software quality – definition
  - Software quality assurance – definition and objectives
  - Software quality assurance and software engineering

OHT 2.3

2.1 Software - IEEE definition

Definition of software is really not simple. Simply code?

According to the IEEE:

- Software is: Computer programs, procedures, and possibly associated documentation and data pertaining to the operation of a computer system.

- **A 'similar definition comes from ISO:**

ISO definition (from ISO 9000-3) lists four components necessary to assure the quality of the software development process and years of maintenance:

- computer programs (code)
- procedures
- documentation
- data necessary for operating the software system.

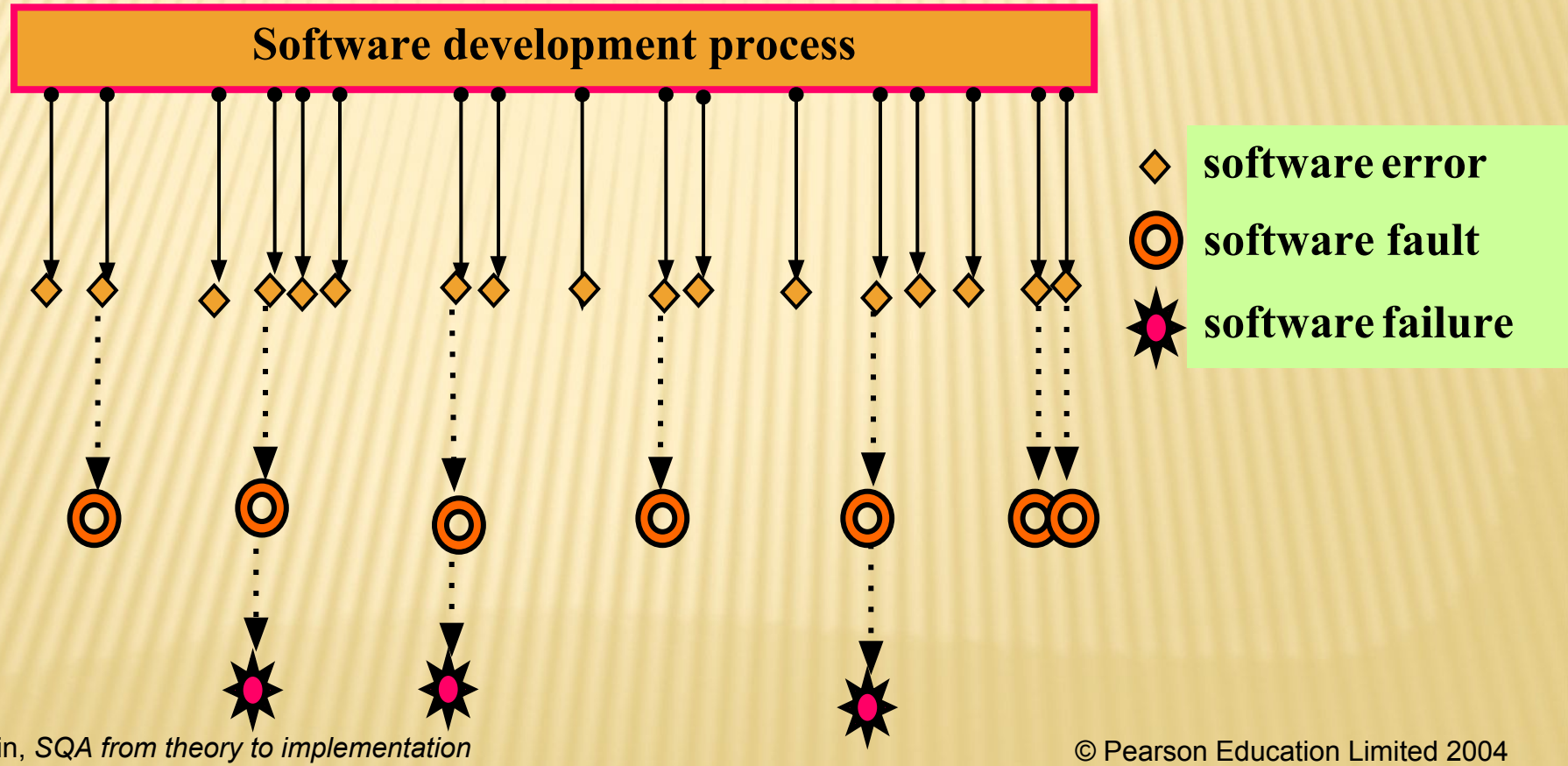
OHT 2.4

BASIC DEFINITIONS

- **Software Error – made by programmer**
  - Syntax (grammatical) error
  - Logic error (multiply vice add two operands)
- **Software Fault –**
  - All software errors may **not** cause software faults
  - That part of the software may not be executed
    - (An error is present but not encountered….)
- **Software Failures – Here's the interest.**
  - A software fault becomes a software failure when/if it is **activated**.
  - Faults may be found in the software due to the way the software is executed or other constraints on the software's execution, such as execution options.
    - Some runs result in **failures**; some **not**.
    - Example: **standard software** running in different client shops.

OHT 2.5

2.2 Software errors, software faults and software failures



OHT 2.6

2.3 THE NINE CAUSES OF SOFTWARE ERRORS

1. Faulty requirements definition

- Usually considered the root cause of software errors
- Incorrect requirement definitions
  - Simply stated, 'wrong' definitions (formulas, etc.)
- Incomplete definitions
  - Unclear or implied requirements
- Missing requirements
  - Just flat-out 'missing.' (e.g. Program Element Code)
- Inclusion of unneeded requirements
  - (many projects have gone amuck for including far too many requirements that will never be used.
  - Impacts budgets, complexity, development time, …

THE NINE CAUSES OF SOFTWARE ERRORS

2. Client-developer communication failures

- Misunderstanding of instructions in requirements documentation
- Misunderstanding of written changes during development.
- Misunderstanding of oral changes during development.
- Lack of attention
  - to client messages by developers dealing with requirement changes and
  - to client responses by clients to developer questions

- Very often, these very talented individuals come from different planets, it seems.
  - Clients represent the users; developers represent a different mindset entirely some times!

THE NINE CAUSES OF SOFTWARE ERRORS

3. Deliberate deviations from software requirements

- Developer reuses previous / similar work to save time.
- Often reused code needs modification which it may not get or contains unneeded / unusable extraneous code.

- Book suggests developer(s) may overtly omit functionality due to time / budget pressures.
  - Another BAD choice; System testing will uncover these problems to everyone's dismay!
  - I have never seen this done intentionally!
- Developer inserting unapproved 'enhancements' (perfective coding; a slick new sort / search….); may also ignore some seemingly minor features, which sometimes are quite major.
  - Have seen this and it too causes problems and embarrassment during reviews.

THE NINE CAUSES OF SOFTWARE ERRORS

4. Logical design errors

- Definitions that represent software requirements by means of erroneous algorithms.
  - Yep! Wrong formulas; Wrong Decision Logic Tables; incorrect text; wrong operators / operands…
- Process definitions: procedures specified by systems analyst not accurate reflection of the business process specified.
  - Note: all errors are not necessarily software errors.
  - This seems like a procedural error, and likely not a part of the software system…
- Erroneous Definition of Boundary Condition – a common source of errors
  - The "absolutes" like 'no more than' "fewer than," "n times or more;" "the first time," etc.

THE NINE CAUSES OF SOFTWARE ERRORS

4. Logical design errors (continued)

- Omission of required software system states
  - If rank is >= O1 and RPI is numeric, then….easy to miss action based on the software system state.

- Omission of definitions concerning reactions to illegal operation of the software system.

  Including code to detect an illegal operation but failure to design the computer software reaction to this:

  Gracefully terminate, sound alarm, etc.

THE NINE CAUSES OF SOFTWARE ERRORS

5. Coding errors

- Too many to try to list.
  - Syntax errors (grammatical errors)
  - Logic errors (program runs; results wrong)
  - Run-time errors (crash during execution)

THE NINE CAUSES OF SOFTWARE ERRORS

6. Non-compliance w/documentation & coding instructions

- Non-compliance with published templates (structure)
- Non-compliance with coding standards (attribute names…)
- (Standards and Integration Branch)
  - Size of program;
    - Other programs must be able to run in environment!
    - Data Elements and Codes: AFM 300-4;
    - Required documentation manuals and operating instructions; AFDSDCM 300-8, etc…
- SQA Team: testing not only execution software but coding standards; manuals, messages displayed; resources needed; resources named (file names, program names,…)

THE NINE CAUSES OF SOFTWARE ERRORS

7. Shortcomings of the Testing Process

- Likely the part of the development process cut short most frequently!
- Incomplete test plans
  - Parts of application not tested or tested thoroughly!
- Failure to document, report detected errors and faults
  - So many levels of testing….we will cover.
- Failure to quickly correct detected faults due to unclear indications that there ‘was’ a fault
- Failure to fix the errors due to time constraints
  - Many philosophies here depending on severity of the error.

THE NINE CAUSES OF SOFTWARE ERRORS

8. User interface and procedure errors

9. Documentation errors

- Errors in the design documents
  - Trouble for subsequent redesign and reuse
- Errors in the documentation within the software for the User Manuals
- Errors in on-line help, if available.
- Listing of non-existing software functions
  - Planned early but dropped; remain in documentation!
- Many error messages are totally meaningless

OHT 2.15

THE NINE CAUSES OF SOFTWARE ERRORS

The nine causes of software errors are:

1. Faulty requirements definition
2. Client-developer communication failures
3. Deliberate deviations from software requirements
4. Logical design errors
5. Coding errors
6. Non-compliance with documentation and coding instructions
7. Shortcomings of the testing process
8. User interface and procedure errors
9. Documentation errors

You should be conversant with these

So let's move on to 'exactly' what we mean by 'Software Quality.'

As you will see, there is no commonly-agreed to definition.

2.4 Software Quality - IEEE definition

Software quality is:

1) **The degree to which a system, component, or process meets specified requirements.**

by Philip Crosby

2) **The degree to which a system, component, or process meets customer or user needs or expectations.**

by Joseph M. Juran

Now, more closely…

OHT 2.18

Software Quality - IEEE definition

Software quality is:

1) The degree to which a system, component, or process meets specified requirements.

Seems to emphasize the specification, assuming the customer has articulated all that is needed in the specs AND that if the specs are met, the customer will be satisfied.

I have found that this is not necessarily the case, that, if fact, often 'austere' systems are first deployed (errors discovered in specs sometimes very serious); customers acquiesce to the deployment with understanding of a follow-on deployment.

OHT 2.19

Software Quality - IEEE definition

Software quality is: (Joseph Juran)

2) **The degree to which a system, component, or process meets customer or user needs or expectations.**

Here, emphasis is on a satisfied customer whatever it takes.
Implies specs may need corrections

But this seems to free the customer from 'professional responsibility' for the accuracy and completeness of the specs!

Assumption is that real needs can be articulated during development. This may occur, but in fact major problems can be discovered quite late. Not a happy customer!

OHT 2.20

Roger Pressman's Definition of Software Quality

Pressman believes that Software quality is :

- **Conformance to explicitly stated functional and performance requirements, (meets the specs)**
  - **Discuss: 'functional' and 'performance' specs!!**
- **Explicitly documented development standards, and**
  seems to imply a documented development process
- **Implicit characteristics that are expected of all professionally developed software.**
  further seems to imply quality as found in reliability, maintainability, scalability, usability, and more

OHT 2.21

2.5 Software Quality Assurance – Various Definitions

Software Quality Assurance is:

1. **A planned and systematic pattern of all actions necessary to provide adequate confidence that an item or product conforms to established technical requirements.**
2. **A set of activities designed to evaluate the process by which the products are developed or manufactured. Contrast with: quality control.**

More closely:

OHT 2.22

SQA - IEEE DEFINITION

Says to plan and implement systematically!

shows progress and instills confidence software is coming along

Refers to a software development process

a methodology; a way of doing things;

Refers to the specification of technical requirements

must have these.

Note that SQA must include not only process for development but for (hopefully) years of maintenance. So, **we need to consider quality issues** affecting **not only development but also maintenance** into overall SQA concept.

SQA activities must also include **scheduling** and **budgeting**.

SQA must address **issues** that arise when time constraints are encountered – are features eliminated? Budget constraints may force **compromise** when/if inadequate resources are allocated to development and/or maintenance.

OHT 2.23

SQA - Expanded Definition

Software quality assurance is:

A systematic, planned set of actions necessary to provide adequate confidence that the software development process or the maintenance process of a software system product conforms to established functional technical requirements as well as with the managerial requirements of keeping the schedule and operating within the budgetary confines.

This SQA definition supports the concept of the ISE 9000 standards
regarding SQA, and corresponds to main outlines of the Capability Maturity Model (CMM) for software.

Our book adapts the **Expanded Definition of SQA**.

See Table 2.2. We will be looking at these a lot later…

OHT 2.25

COMPARISON WITH ISO 9000-3 AND SEI-CMM

- Table 2 compares the elements of the **expanded SQA definition** with the relevant sections of both the **ISO-9000-3** and the software **CMM**.
- We will discuss these in depth coming up.

OHT 2.26

2.5.2 SOFTWARE QUALITY ASSURANCE VS. SOFTWARE QUALITY CONTROL

DIFFERENT OBJECTIVES.

- **Quality Control** is defined as a designed to evaluate the quality of a set of **activities** developed or manufactured product
  - We have QC inspections during development and before deployment
  - QC activities are only a **part** of the total range of QA activities.
- **Quality Assurance's** objective is to minimize the **cost of guaranteeing quality** by a **variety of activities performed throughout the development / manufacturing processes / stages.**
  - Activities prevent causes of errors; detect and correct them early in the development process
  - QA substantially reduces the rate of products that do not qualify for shipment and/at the same time, reduce the costs of guaranteeing quality in most cases.

OHT 2.27

2.5.3 The objectives of SQA activities in Software Development (Process-Oriented

1) Assuring an acceptable level of confidence that the software will conform to functional technical requirements.

2) Assuring an acceptable level of confidence that the software will conform to managerial scheduling and budgetary requirements.

(3) Initiation and management of activities for the improvement and greater efficiency of software development and SQA activities.

OHT 2.28

The objectives of SQA activities in Software Maintenance (product-oriented)

1) **Assuring an acceptable level of confidence that the software maintenance activities will conform to the functional technical requirements.**

2) **Assuring an acceptable level of confidence that the software maintenance activities will conform to managerial scheduling and budgetary requirements.**

(3) **Initiate and manage activities to improve and increase the efficiency of software maintenance and SQA activities.**

OHT 2.29

HOMEWORK ASSIGNMENT

You are to answer the following questions in essay format and send to me via Blackboard Assignment Chapter 2:

Question 2.5
Question 2.7

OHT 2.30

TEAM DISCUSSION

Team 2 will fully discuss the following questions in our next class:

Questions: 2.2
2.3
2.4

On page 34.